

Resumen Técnico de Solución – Jhonatan Saldarriaga

Objetivo

Diseñar y desplegar un pipeline de datos robusto y modular utilizando Apache Airflow, con el propósito de construir un dataset enriquecido que integre eventos de impresiones, clics y pagos por usuario y value_prop. El objetivo es habilitar una base sólida para procesos analíticos y desarrollo de modelos de machine learning.

Stack Tecnológico y Decisiones de Arquitectura

Componente	Justificación
Apache Airflow (Dockerizado)	Orquestación de tareas, manejo de dependencias y reintentos automáticos
Python 3.8	Lenguaje principal por su ecosistema y capacidad de manipulación de datos
PostgreSQL	Base de datos relacional confiable para persistencia del dataset final
Pandas, SQLAlchemy	Manipulación eficiente de DataFrames y conexión con BD
Docker + Docker Compose	Aislamiento del entorno de ejecución y despliegue reproducible
Jupyter Notebook	Exploración visual y análisis exploratorio de datos

Estructura del DAG en Airflow y Pipeline

- **start**: Inicializa la ejecución del pipeline con trazabilidad.
- **extract_data**: Carga y normaliza los datasets desde fuentes JSON y CSV, y los almacena en XCom.
- **transform_data**: Enriquecimiento del dataset con lógica temporal (última semana vs 3 semanas anteriores).
- **validate_data**: Revisión de duplicados y nulos como validación de integridad previa al guardado.
- **save_outputs**: Persistencia del dataset en CSV, Parquet y PostgreSQL. Se utiliza `replace` para entornos de prueba.
- **end**: Cierre del DAG con logging de finalización.

Exploración de Datos

Se diseñó un notebook en Jupyter con análisis visuales y estadísticos sobre el dataset generado, incluyendo distribuciones por value_prop, usuarios más activos, y correlaciones con la variable de clics. Este análisis validó la consistencia del pipeline y la utilidad del dataset para tareas de modelado.

Buenas Prácticas Aplicadas

- Modularización clara del código por funcionalidad (`load\_data`, `transform\_data`, `save\_outputs`, etc.).
- Logging informativo con contexto en cada paso del pipeline.
- Validaciones proactivas de schema, columnas clave y estructuras de datos.
- Tolerancia a errores y diseño reproducible vía Docker.
- Separación de responsabilidades entre lógica de negocio y de infraestructura.